

Diagrammes UML

Diagramme de cas d'utilisation & diagramme de classes

Module de gestion des bénévoles et des événements

Extension du site web vitrine de Terra Sana ASBL

Document complémentaire au cahier des charges (déjà transmis). Il rassemble les deux diagrammes demandés ainsi qu'une analyse détaillée, en particulier pour le diagramme de classes.

Encadreur scolaire : Marie-Christine Namur

Maître de stage : Didier Seraye

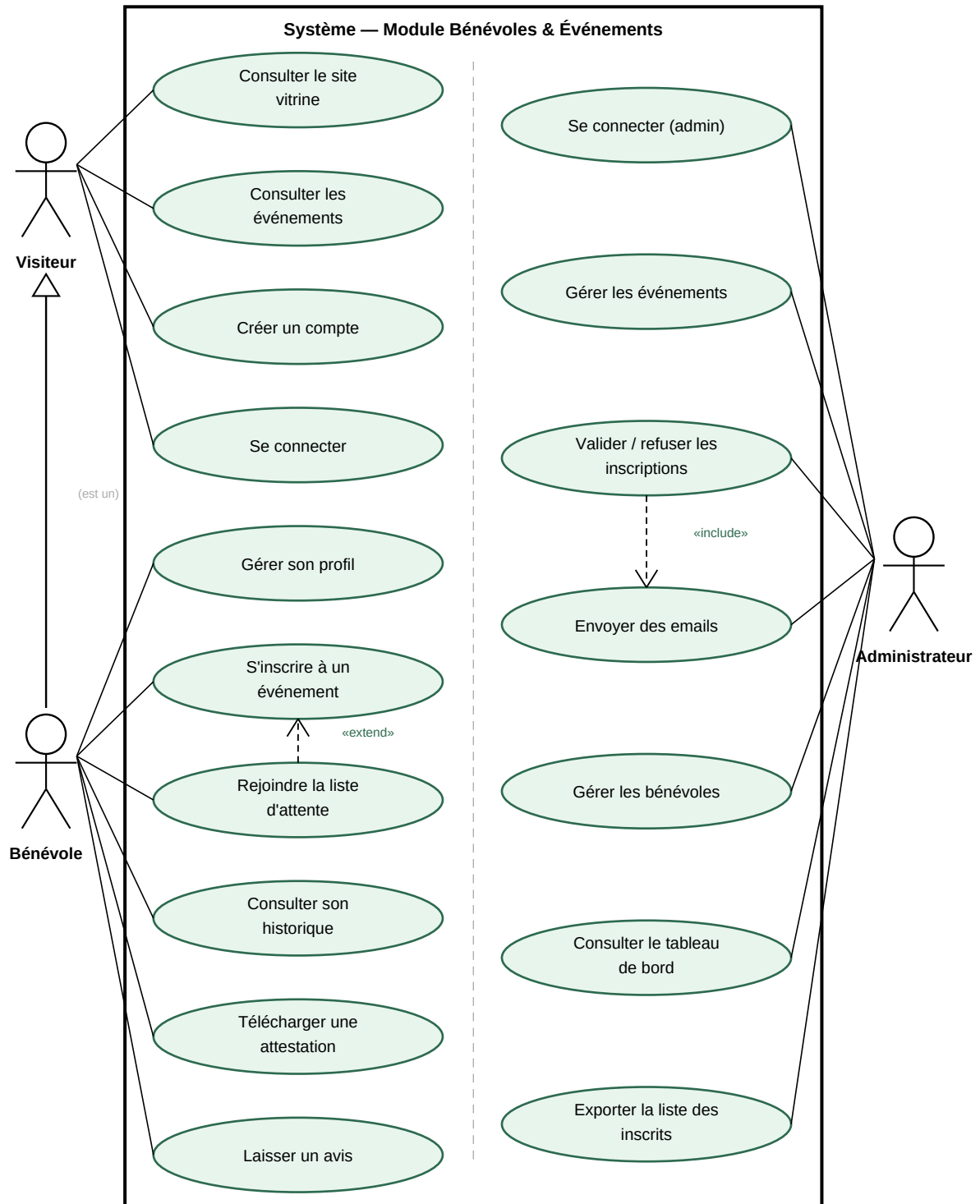
Travail présenté par Youndjeu Tchouapi Alain

EAFC Uccle

2025-2026

1. Diagramme de cas d'utilisation

Le diagramme ci-dessous décrit les interactions entre les acteurs et le module. Il distingue le périmètre côté bénévole du périmètre d'administration, tout en restant dans une seule frontière système puisqu'il s'agit d'une même application.



1.1 Les acteurs

Trois acteurs interagissent avec le module :

- **Visiteur** — internaute non authentifié. Il peut consulter le site vitrine et la liste des événements, créer un compte et se connecter.
- **Bénévole** — visiteur qui s'est authentifié. Le lien de **généralisation** (flèche à triangle creux) indique qu'un bénévole *est un* visiteur : il hérite de tous ses cas d'utilisation et y ajoute les siens (gestion du profil, inscription aux événements, historique, attestation, avis).
- **Administrateur** — membre de Terra Sana qui gère le module depuis l'espace d'administration (événements, inscriptions, communication, tableau de bord).

1.2 Les relations «include» et «extend»

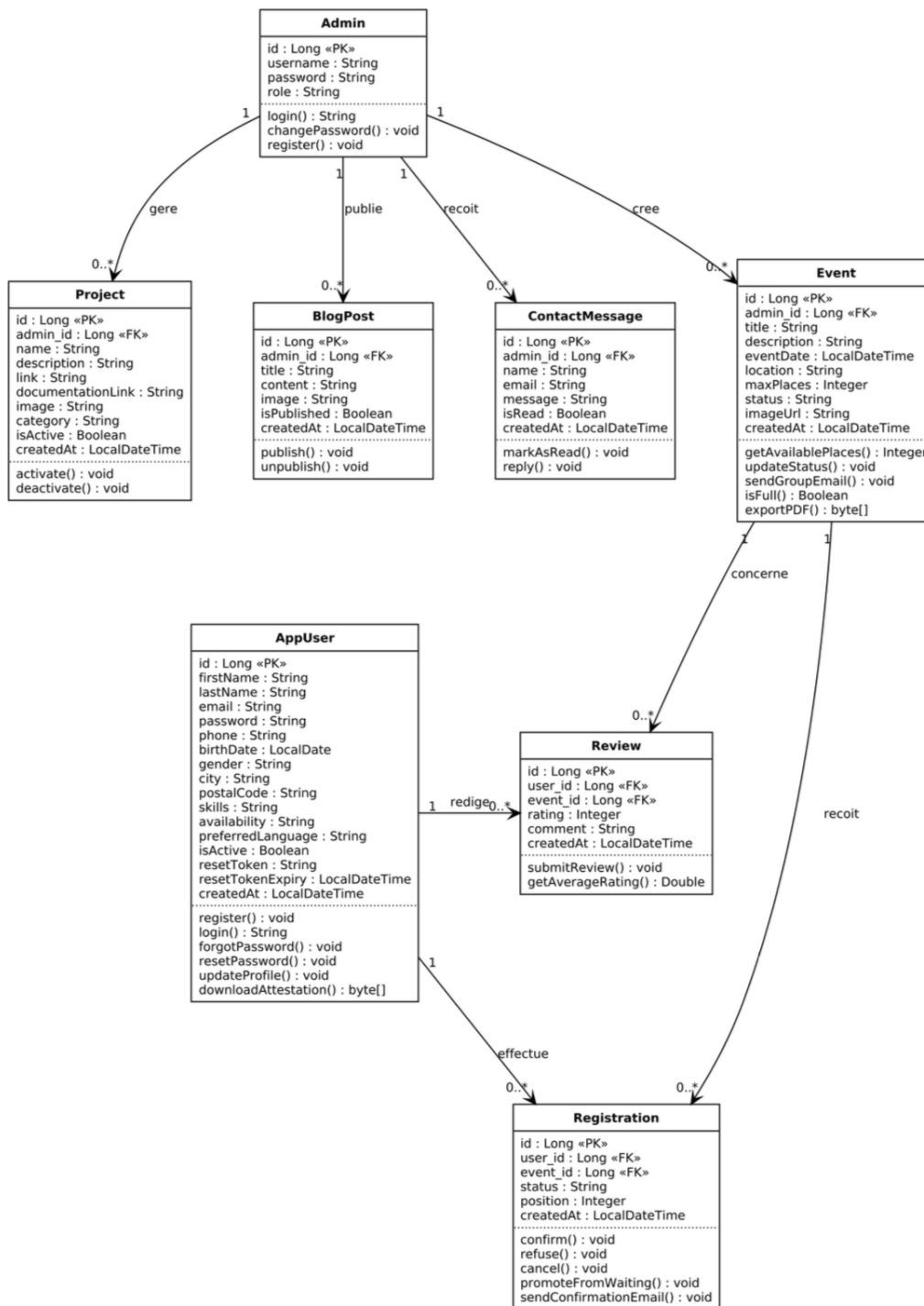
Deux relations de dépendance enrichissent le modèle et traduisent des règles métier concrètes :

- **«extend»** — « Rejoindre la liste d'attente » **étend** « S'inscrire à un événement ». C'est un comportement *optionnel*, déclenché uniquement sous condition : lorsque l'événement a atteint son nombre maximum de places, le bénévole est basculé en liste d'attente au lieu d'être inscrit directement.
- **«include»** — « Valider / refuser les inscriptions » **inclut** « Envoyer des emails ». L'envoi d'une notification au bénévole concerné fait *systématiquement* partie du traitement : il n'y a pas de validation ou de refus sans email de confirmation. La relation « include » exprime précisément ce comportement obligatoire et réutilisable.

Rappel de notation : trait plein + triangle creux = généralisation ; flèche en pointillés + pointe ouverte = dépendance «include» / «extend».

2. Diagramme de classes

Le diagramme présente les huit entités du modèle de données avec leurs attributs (clés primaires «PK» et étrangères «FK»), leurs méthodes et leurs associations nommées. AppUser, Event, Review et Registration sont créées dans le cadre du TFE ; Admin, Project, BlogPost et ContactMessage proviennent du site existant.



2.1 Rôle de chaque classe

Classe	Rôle dans le module
AppUser	Représente un bénévole. Entité centrale du TFE : porte l'identité et le profil complet (coordonnées, compétences, disponibilités), l'authentification (connexion, mot de passe oublié / réinitialisé) et le téléchargement de l'attestation.
Event	Un événement organisé par Terra Sana : date, lieu, nombre de places, statut (ouvert / complet / clôturé), visuel. Seconde entité centrale du module.
Registration	Entité associative entre AppUser et Event (deux clés étrangères user_id et event_id). Elle matérialise l'inscription et porte ses propres attributs : statut (confirmée / en attente / refusée) et position dans la file d'attente.
Review	Un avis laissé par un bénévole sur un événement auquel il a participé (note de 1 à 5 et commentaire).
Admin	Compte d'administration qui gère l'ensemble du module et les contenus existants du site.

2.2 Attributs, clés et méthodes

- **Clés primaires et étrangères** — chaque classe possède un identifiant `id` : Long «PK». Les stéréotypes «FK» (`admin_id`, `user_id`, `event_id`) matérialisent les liens vers les autres tables : le diagramme se traduit ainsi directement en schéma relationnel MySQL.
- **Trois compartiments** — chaque classe est découpée en nom, attributs puis méthodes. Les méthodes traduisent les traitements métier : `confirm()`, `refuse()`, `promoteFromWaiting()` et `sendConfirmationEmail()` sur `Registration` ; `getAvailablePlaces()`, `isFull()` et `exportPDF()` sur `Event` ; `downloadAttestation()` sur `AppUser`.
- **Sécurité (implémentation)** — les mots de passe ne sont jamais stockés en clair (hachage BCrypt côté Spring Security) et l'email de `AppUser` est unique en base ; ces règles sont portées par les contraintes de la base et la couche service.

2.3 Associations et cardinalités

Chaque association est nommée par un verbe métier et porte ses cardinalités. Associations propres au module TFE :

Association	Verbe	Card.	Lecture
AppUser → Registration	effectue	1 → 0..*	Un bénévole effectue plusieurs inscriptions ; chaque inscription appartient à un seul bénévole.
Event → Registration	reçoit	1 → 0..*	Un événement reçoit plusieurs inscriptions ; chaque inscription concerne un seul événement.
AppUser → Review	rédige	1 → 0..*	Un bénévole peut rédiger plusieurs avis.
Review → Event	concerne	0..* → 1	Un avis concerne un seul événement ; un même événement peut en recevoir plusieurs.

L'administrateur, lui, pilote aussi bien les nouvelles entités que les contenus existants : il **crée** les Event (1 → 0..*) et **gère** / **publie** / **reçoit** respectivement les Project, BlogPost et ContactMessage du site.

La classe **Registration** est le pivot du modèle : placée entre AppUser et Event avec ses deux clés étrangères, elle transforme une relation « plusieurs à plusieurs » (un bénévole s'inscrit à plusieurs événements, un événement accueille plusieurs bénévoles) en deux relations « un à plusieurs » exploitables, tout en offrant un emplacement naturel pour stocker le statut et la position en liste d'attente.

2.4 Choix de conception

- **Réutilisation de l'existant** — le compte Admin déjà présent pilote aussi bien les nouvelles entités que les contenus du site (Project, BlogPost, ContactMessage). On étend la base sans la réécrire ni dupliquer un compte de gestion.
- **Traçabilité PK / FK** — l'affichage explicite des clés primaires et étrangères rend le passage au modèle relationnel MySQL immédiat : chaque «FK» correspond à une contrainte de clé étrangère réelle en base.
- **Cohérence avec le code** — chaque classe correspond à une entité JPA (Spring Boot) et à une table MySQL ; les attributs, types et méthodes reflètent directement l'implémentation.

Les deux diagrammes sont cohérents entre eux : chaque cas d'utilisation manipule une ou plusieurs de ces classes (« S'inscrire à un événement » crée une Registration, « Laisser un avis » crée une Review, « Gérer les événements » agit sur Event, etc.).